

# Java 25 para programadores Kotlin

2026

## Contents

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Java para programadores Kotlin</b> | <b>2</b>  |
| 1.1      | Tipos de datos                        | 3         |
| 1.2      | Operadores                            | 4         |
| 1.3      | Estructuras de control de flujo       | 5         |
| 1.4      | Arrays                                | 8         |
| 1.5      | Métodos y Funciones                   | 9         |
| 1.5.1    | Métodos en Java                       | 9         |
| 1.5.2    | Funciones en Kotlin                   | 9         |
| 1.5.3    | Funciones lambda                      | 10        |
| 1.5.4    | Funciones de una sola línea           | 10        |
| 1.6      | Orientación a objetos                 | 11        |
| 1.6.1    | Clases                                | 11        |
| 1.6.2    | Objetos                               | 12        |
| 1.6.3    | Paquetes                              | 12        |
| 1.7      | Orientación a objetos avanzada        | 13        |
| 1.7.1    | Encapsulamiento                       | 13        |
| 1.7.2    | Getters y Setters                     | 14        |
| 1.7.3    | Herencia                              | 15        |
| 1.7.4    | Polimorfismo                          | 15        |
| 1.7.5    | Interfaces                            | 16        |
| 1.7.6    | Clases abstractas                     | 17        |
| 1.7.7    | Sobrecarga de métodos                 | 18        |
| 1.7.8    | Sobrescritura de métodos              | 18        |
| <b>2</b> | <b>Java SE (Standard Edition)</b>     | <b>19</b> |
| 2.1      | Entrada y salida de datos             | 19        |
| 2.1.1    | Entrada                               | 19        |
| 2.1.2    | Salida                                | 20        |
| 2.2      | Excepciones                           | 21        |
| 2.2.1    | Excepciones a medida                  | 22        |
| 2.3      | Archivos y directorios                | 22        |
| 2.4      | Streams                               | 23        |
| 2.5      | Colecciones                           | 24        |
| 2.5.1    | Operaciones de colección              | 25        |
| 2.6      | Multithreading                        | 26        |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| 2.7      | Genéricos                                                   | 27        |
| <b>3</b> | <b>Java 25: qué es nuevo y qué enseñar</b>                  | <b>28</b> |
| 3.1      | Características permanentes de Java 25                      | 28        |
| 3.2      | Características modernas anteriores que sí conviene conocer | 28        |
| 3.3      | Preview de Java 25                                          | 28        |
| 3.4      | Lo que no se debe presentar como Java 25                    | 28        |
| <b>4</b> | <b>Java web: frontera de esta guía</b>                      | <b>29</b> |
| <b>5</b> | <b>Actividades propuestas</b>                               | <b>29</b> |
| 5.1      | Actividad 1: traducir el modelo                             | 29        |
| 5.2      | Actividad 2: resultado sellado                              | 29        |
| 5.3      | Actividad 3: colección idiomática                           | 29        |
| 5.4      | Actividad 4: frontera null                                  | 29        |
| 5.5      | Actividad 5: concurrencia con criterio                      | 29        |
| <b>6</b> | <b>Referencias oficiales</b>                                | <b>29</b> |

## 1 Java para programadores Kotlin

En este documento vamos a realizar una introducción rápida a Java para programadores que ya saben Kotlin. Veremos las principales diferencias entre ambos lenguajes y cómo pasar de uno a otro.

Java es un lenguaje de programación de propósito general, desarrollado a mediados de los años 90. Es uno de los lenguajes más populares y ampliamente utilizados en la industria. Aunque Kotlin es un lenguaje relativamente nuevo, ha ganado mucha popularidad debido a su simplicidad y conveniencia para desarrollar aplicaciones para Android. Si ya sabes Kotlin, aprender Java debería ser relativamente sencillo para ti, ya que ambos lenguajes tienen muchas similitudes.

Algunas de las características principales de Java son:

- Es un lenguaje orientado a objetos: todos los elementos en Java son objetos o instancias de clases (aunque también tiene tipos primitivos, como veremos).
- Es un lenguaje de tipado estático: esto significa que se deben declarar los tipos de datos de las variables y métodos en tiempo de compilación.
- Tiene una sintaxis similar a la de C y C++.

Aquí hay algunos ejemplos de cómo se vería una clase en Java y Kotlin:

Java:

```
public class MiClase {
    private int miVariable;

    public MiClase(int miVariable) {
        this.miVariable = miVariable;
    }

    public void miMetodo() {
```

```

        System.out.println("Hola mundo!");
    }
}

```

Kotlin:

```

class MiClase(private val miVariable: Int) {
    fun miMetodo() {
        println("Hola mundo!")
    }
}

```

Como puedes ver, ambas clases tienen una variable privada y un método público. La sintaxis es ligeramente diferente, pero el concepto es el mismo. En Java hay que escribir explícitamente el constructor, el tipo de retorno y las palabras clave `public`, `private`, etc. Kotlin omite mucho de eso por convención.

## 1.1 Tipos de datos

Java y Kotlin tienen muchos tipos de datos similares, pero también hay algunas diferencias importantes.

En Java existen dos categorías de tipos: los **tipos primitivos** (`int`, `long`, `double`, `float`, `boolean`, `char`, `byte`, `short`) y los **tipos referencia** (`String`, cualquier clase, arrays, colecciones...). Los tipos primitivos no son objetos: tienen un tamaño fijo en memoria y no tienen métodos. Cuando necesitas usar un primitivo como objeto (por ejemplo, en una colección genérica), Java realiza automáticamente un **boxing**: convierte `int` en `Integer`, `boolean` en `Boolean`, etc.

En Kotlin, todos estos tipos están disponibles, pero se escriben con la primera letra mayúscula: `Int`, `Long`, `Double`, `Boolean`... Aunque en la JVM el compilador usa primitivas cuando es posible, en el código Kotlin se ven como clases.

Aquí hay un ejemplo de cómo se vería la declaración de variables en Java y Kotlin:

En Java las declaraciones siguen la estructura:

```

tipoDeDato nombreDeVariable = valorInicial;

```

Por ejemplo:

```

int miEntero = 5;
double miFlotante = 3.14;
char miCaracter = 'a';
boolean miBoolean = true;
String miCadena = "Hola mundo";

```

En Kotlin, la sintaxis para declarar variables es un poco diferente:

```

var nombreDeVariable: TipoDeDato = valorInicial

```

Ejemplos:

```

var miEntero: Int = 5
var miFlotante: Double = 3.14
var miCaracter: Char = 'a'
var miBoolean: Boolean = true

```

```
var miCadena: String = "Hola mundo"
```

En ambos casos, se están declarando variables de distintos tipos y se les está asignando un valor inicial. En Java, se deben especificar explícitamente los tipos de datos de las variables. En Kotlin, el compilador puede inferir los tipos de datos a partir de los valores iniciales, así que `var miEntero = 5` funciona sin escribir `: Int`.

En Java existe la inferencia local con `var` desde Java 10, pero no es equivalente a `val` o `var` de Kotlin: no puede usarse en campos, parámetros ni retornos, y la variable sigue siendo mutable.

## 1.2 Operadores

Java y Kotlin tienen muchos operadores similares, como los operadores aritméticos (+, -, \*, /, %), los operadores de asignación (, +, -, etc.), los operadores de comparación (=, !=, >, <, etc.) y los operadores lógicos (&&, ||, !).

Aquí hay algunos ejemplos de cómo se utilizan estos operadores en Java y Kotlin:

Java:

```
int a = 5;
int b = 2;
int c = a + b; // c es igual a 7
int d = a - b; // d es igual a 3
int e = a * b; // e es igual a 10
int f = a / b; // f es igual a 2
int g = a % b; // g es igual a 1

a += b; // a es igual a 7
b -= a; // b es igual a -5

boolean h = a > b; // h es igual a true
boolean i = a == b; // i es igual a false
```

En Kotlin los operadores aritméticos y de comparación son iguales, pero hay algunos operadores que no están disponibles en Java:

- **Operador Elvis (?):** evalúa si una variable es `null` o no. Si la variable es `null`, devuelve un valor por defecto. En Java se simula con el operador ternario.

```
val a: String? = null
val b = a ?: "valor por defecto" // b es igual a "valor por defecto"
```

```
String a = null;
String b = (a != null) ? a : "valor por defecto";
```

- **Operador de rango (..):** crea un rango de valores entre dos operandos. En Java se puede simular con un bucle o con `IntStream.rangeClosed()`.

```
val rango = 1..10 // [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

- **Operador de deestructuración:** descompone un objeto en sus componentes. Funciona con `data class` y con cualquier clase que implemente los métodos `componentN`.

```
data class MiClase(val a: Int, val b: String)
val objeto = MiClase(1, "Hola")
val (a, b) = objeto // a es igual a 1 y b es igual a "Hola"
```

- **Operador in:** comprueba si un elemento se encuentra en un rango o en una colección. En Java se usa el método `contains`.

```
if (x in 1..10) {
    // x está en el rango de 1 a 10
}
```

```
if (x >= 1 && x <= 10) {
    // x está en el rango de 1 a 10
}
```

- **Operador is:** comprueba si un objeto es de un determinado tipo. En Java se usa `instanceof`.

```
if (obj is String) {
    println(obj.length) // smart cast: obj ya es String aquí
}
```

```
if (obj instanceof String) {
    String s = (String) obj; // hay que hacer cast manual
    System.out.println(s.length());
}
```

Desde Java 16, el pattern matching con `instanceof` permite la asignación directa:

```
if (obj instanceof String s) {
    System.out.println(s.length());
}
```

### 1.3 Estructuras de control de flujo

Las estructuras de control son elementos de programación que controlan el flujo de ejecución de un programa. Algunas de las estructuras de control más comunes en Java y Kotlin son:

**Bloques if-else:** Permiten tomar decisiones en función de si una condición es verdadera o falsa. En Java, se pueden escribir de la siguiente manera:

```
if (condition) {
    // código a ejecutar si la condición es verdadera
} else {
    // código a ejecutar si la condición es falsa
}
```

En Kotlin, la sintaxis es prácticamente idéntica:

```
if (condition) {
    // código a ejecutar si la condición es verdadera
} else {
    // código a ejecutar si la condición es falsa
}
```

La diferencia principal es que en Kotlin `if` es una **expresión**: puede devolver un valor. En Java, `if` es una **sentencia** y no devuelve nada; para obtener un valor equivalente se usa el operador ternario

==? : :

```
val mensaje = if (esMayor) "Mayor" else "Menor"
```

```
String mensaje = esMayor ? "Mayor" : "Menor";
```

**Ciclos for:** Permiten ejecutar un bloque de código repetidamente un número determinado de veces. En Java, se pueden escribir de la siguiente manera:

```
for (int i = 0; i < 10; i++) {  
    // código a ejecutar en cada iteración del ciclo  
}
```

En Kotlin:

```
for (i in 0 until 10) {  
    // código a ejecutar en cada iteración del ciclo  
}
```

Kotlin también ofrece rangos avanzados: `1..10` incluye el 10, `1 until 10` lo excluye. Java no tiene esta distinción de sintaxis.

**Ciclos while:** Permiten ejecutar un bloque de código mientras se cumpla una condición. En Java:

```
while (condition) {  
    // código a ejecutar mientras se cumpla la condición  
}
```

En Kotlin la sintaxis es igual:

```
while (condition) {  
    // código a ejecutar mientras se cumpla la condición  
}
```

**Ciclos do-while:** Son similares a los ciclos while, pero aseguran que el bloque de código se ejecute al menos una vez. En Java:

```
do {  
    // código a ejecutar al menos una vez  
} while (condition);
```

En Kotlin la diferencia es solo que no lleva punto y coma al final:

```
do {  
    // código a ejecutar al menos una vez  
} while (condition)
```

**Estructuras de flujo:** `break` interrumpe el ciclo más interno, `continue` salta a la siguiente iteración, y `return` finaliza la función. Funcionan igual en ambos lenguajes:

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) break;    // Interrumpe el ciclo  
    if (i % 2 == 0) continue; // Salta a la siguiente iteración  
    System.out.println(i);  
}
```

```
for (i in 0 until 10) {  
    if (i == 5) break  
    if (i % 2 == 0) continue
```

```
    println(i)
}
```

**switch-case:** Permiten tomar decisiones basadas en el valor de una variable. En Java, se pueden escribir de la siguiente manera:

```
int value = 1;

switch (value) {
    case 1:
        System.out.println("Uno");
        break;
    case 2:
        System.out.println("Dos");
        break;
    default:
        System.out.println("Otro");
}
```

En Kotlin se usa **when**, que es más flexible y no necesita **break**:

```
val value = 1

when (value) {
    1 -> println("Uno")
    2 -> println("Dos")
    else -> println("Otro")
}
```

Desde Java 14 también existen las expresiones **switch** con flechas, que no necesitan **break**:

```
String resultado = switch (value) {
    case 1 -> "Uno";
    case 2 -> "Dos";
    default -> "Otro";
};
```

**try-catch:** Permiten manejar excepciones (errores en tiempo de ejecución) de manera controlada. En Java:

```
try {
    // código que puede lanzar una excepción
} catch (Exception e) {
    // código a ejecutar si se lanza una excepción
} finally {
    // código que se ejecuta siempre
}
```

En Kotlin la sintaxis es similar:

```
try {
    // código que puede lanzar una excepción
} catch (e: Exception) {
    // código a ejecutar si se lanza una excepción
}
```

```
} finally {  
    // código que se ejecuta siempre  
}
```

## 1.4 Arrays

En Java y Kotlin los arrays son similares en muchos aspectos, pero hay algunas diferencias notables.

Java tiene una clase especial llamada `java.util.Arrays` que proporciona métodos útiles para trabajar con arrays, como `sort()` para ordenar elementos y `binarySearch()` para buscar un elemento en un array ordenado. Los arrays en Java son mutables: se pueden modificar después de su creación, pero tienen un tamaño fijo.

En Kotlin, los arrays también tienen tamaño fijo y son mutables. La diferencia es la sintaxis: Kotlin ofrece funciones como  `arrayOf()`, `intArrayOf()`, etc. Además, Kotlin tiene colecciones más cómodas que los arrays para la mayoría de casos.

Ejemplo en Java:

```
// Declarar un array de enteros con tamaño fijo de 5 elementos  
int[] intArray = {1, 2, 3, 4, 5};  
  
// Acceder a un elemento específico del array  
System.out.println(intArray[2]); // imprimiría 3  
  
// Modificar un elemento específico del array  
intArray[3] = 6;  
  
// Recorrer un array usando un for loop  
for (int i : intArray) {  
    System.out.println(i);  
}
```

En Kotlin:

```
// Declarar un array de enteros con tamaño fijo  
val intArray = intArrayOf(1, 2, 3, 4, 5)  
  
// Acceder a un elemento específico del array  
println(intArray[2]) // imprimiría 3  
  
// Modificar un elemento específico del array  
intArray[3] = 6  
  
// Recorrer un array usando un for loop  
for (i in intArray) {  
    println(i)  
}
```

Nota importante: en Kotlin, `val` sobre un array solo impide reasignar la referencia, no modifica sus elementos. Para listas de tamaño variable usa `mutableListOf()` en lugar de arrays.

## 1.5 Métodos y Funciones

En Java y en Kotlin, las funciones son bloques de código que realizan una tarea específica y pueden ser llamadas desde otras partes del programa. Ambos lenguajes tienen algunas similitudes y diferencias en cómo se definen y se utilizan las funciones.

### 1.5.1 Métodos en Java

En Java, un método es una sección de código que se puede llamar desde otra parte de una clase. Los métodos están siempre dentro de una clase (no existen las funciones sueltas) y pueden tener parámetros de entrada y devolver un valor. La sintaxis para definir un método es:

```
<modificadores de acceso> <tipo de retorno> <nombre del método>(<parámetros>) {  
    <cuerpo del método>  
}
```

Por ejemplo, un método que suma dos enteros:

```
class MathUtils {  
    public static int sum(int a, int b) {  
        return a + b;  
    }  
}  
  
// Llamada al método (desde otra parte de la clase o desde main)  
int resultado = MathUtils.sum(1, 2);
```

Como ves, hay que indicar `public static` (o el modificador que corresponda), el tipo de retorno y los parámetros con su tipo. En Java, si no se declara `static`, el método pertenece a una instancia de la clase y necesita un objeto para ser llamado.

### 1.5.2 Funciones en Kotlin

En Kotlin, una función es una sección de código que se puede llamar desde cualquier parte del programa. A diferencia de Java, las funciones en Kotlin no están vinculadas a una clase en particular, y pueden ser definidas tanto dentro como fuera de las clases. La sintaxis es:

```
fun <nombre de la función>(<parámetros>): <tipo de retorno> {  
    <cuerpo de la función>  
}
```

Por ejemplo:

```
fun sum(a: Int, b: Int): Int {  
    return a + b  
}  
  
// Llamada a la función  
val resultado = sum(1, 2)
```

Como se puede ver, ambas formas son bastante similares. En Java, la función se declara con el modificador de acceso `public`, el tipo de valor de retorno `int`, y los parámetros de entrada `int a` y `int b`. En Kotlin, la función se declara con la palabra clave `fun`, seguida del nombre de la función y de los parámetros de entrada, y el tipo de valor de retorno se especifica después de dos puntos (`Int`). En ambos casos, el cuerpo de la función se encierra entre llaves.

Hay algunas diferencias más entre Java y Kotlin en cuanto a funciones. Por ejemplo, en Kotlin se pueden utilizar funciones de una sola línea sin necesidad de usar llaves:

```
fun sum(a: Int, b: Int): Int = a + b
```

En Java no existe esta sintaxis; siempre hay que usar llaves.

Kotlin también permite parámetros con valores por defecto y argumentos nombrados:

```
fun greet(name: String, times: Int = 1) {  
    repeat(times) { println("Hola $name") }  
}  
greet("Ada")           // usa el valor por defecto times = 1  
greet("Ada", 3)        // imprime "Hola Ada" tres veces  
greet(times = 3, name = "Ada") // argumentos nombrados
```

En Java no existe esto de forma nativa; hay que usar sobrecarga de métodos o un builder.

### 1.5.3 Funciones lambda

Las funciones lambda son un tipo especial de funciones anónimas que se pueden utilizar para simplificar el código y hacerlo más conciso. En ambos lenguajes, las funciones lambda se escriben como una expresión que toma unos parámetros de entrada y devuelve un resultado.

En Java, las funciones lambda se introdujeron en la versión 8 y se utilizan con interfaces funcionales, que son interfaces que tienen solo un método abstracto. Para utilizar una función lambda en Java, se necesita una interfaz funcional que declare el tipo de la función:

```
// Declaración de la interfaz funcional  
@FunctionalInterface  
interface Operacion {  
    int ejecutar(int a, int b);  
}  
  
// Declaración de la función lambda y su uso  
Operacion sumar = (int a, int b) -> a + b;  
int resultado = sumar.ejecutar(1, 2);
```

En Kotlin, las funciones lambda se pueden utilizar de forma muy similar, pero con una sintaxis más concisa. En lugar de usar una interfaz funcional, se puede utilizar un tipo de función:

```
val sumar: (Int, Int) -> Int = { a, b -> a + b }  
val resultado = sumar(1, 2)
```

En ambos casos, la función lambda toma dos parámetros de tipo `Int` y devuelve un resultado de tipo `Int`. Las funciones lambda son muy útiles para simplificar el código y hacerlo más legible, especialmente cuando se trata de utilizar código de terceros o librerías externas.

Las interfaces funcionales Java más útiles se encuentran en `java.util.function`: `Function<T, R>` para transformaciones, `Predicate<T>` para filtros, `Consumer<T>` para acciones sin retorno. Kotlin tiene tipos de función propios y puede adaptar interfaces funcionales Java al interoperar con la JVM.

### 1.5.4 Funciones de una sola línea

En Kotlin, es posible definir funciones que consten solo de una línea de código:

```
fun sumar(a: Int, b: Int): Int = a + b

fun multiplicar(a: Int, b: Int): Int {
    return a * b
}
```

Ambas funciones tienen el mismo propósito, pero se han escrito de forma diferente. La función `sumar` es más concisa y legible, mientras que la función `multiplicar` tiene más flexibilidad y puede realizar operaciones más complejas.

En Java no existe la sintaxis de función de una sola línea; siempre necesitas llaves y la palabra `return` (excepto para expresiones lambda).

## 1.6 Orientación a objetos

Java y Kotlin son lenguajes de programación orientados a objetos, lo que significa que utilizan el concepto de "objetos" para modelar elementos del mundo real en el código.

### 1.6.1 Clases

En ambos lenguajes, las clases son plantillas para crear objetos.

En Java, una clase se define con la palabra clave `class` y puede tener constructores, métodos y variables de instancia. Los métodos y variables de instancia son accesibles por medio de objetos creados a partir de la clase:

```
public class Person {
    // Atributos de la clase
    private String name;
    private int age;

    // Constructor de la clase
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Métodos de la clase
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }
}
```

```

    public void setAge(int age) {
        this.age = age;
    }
}

```

En Kotlin, una clase también se define con `class`, pero Kotlin proporciona características adicionales que simplifican la creación de clases. El constructor principal puede incluir directamente los parámetros:

```

class Person(var name: String, var age: Int)

```

Esta una línea en Kotlin equivale a toda la clase Java anterior: Kotlin genera automáticamente los campos, el constructor, getters y setters. Si los parámetros llevan `val`, solo se genera getter (propiedad de solo lectura). Si llevan `var`, se generan getter y setter.

Puedes personalizar el comportamiento con propiedades personalizadas:

```

class Person(var name: String, var age: Int) {
    init {
        require(age >= 0) { "La edad no puede ser negativa" }
    }
}

```

### 1.6.2 Objetos

Los objetos son una de las principales características de la programación orientada a objetos (OOP). Un objeto es una instancia de una clase, y tiene un estado (atributos o variables de instancia) y comportamiento (métodos).

En Java, se crean objetos utilizando la palabra clave `new` seguida del nombre de la clase y los parámetros del constructor:

```

Person persona = new Person("Ada", 30);
persona.setName("Grace");
persona.getName();

```

En Kotlin, se crean objetos de manera similar, pero se omite la palabra clave `new`:

```

val persona = Person("Ada", 30)
persona.name = "Grace"
persona.name

```

En resumen, en ambos lenguajes se crean objetos de una clase y se pueden acceder y modificar sus variables mediante el operador de acceso `..`. Kotlin simplemente simplifica la creación omitiendo `new` y haciendo más corto el acceso a propiedades.

### 1.6.3 Paquetes

Los paquetes son una forma de organizar y agrupar clases y funciones. Ambos lenguajes ofrecen una manera de organizar el código en unidades más pequeñas y manejables para facilitar la reutilización.

En Java, un paquete es una colección de clases y otros archivos de recursos relacionados. Los paquetes se utilizan para organizar el código en categorías lógicas y para evitar conflictos de nombres. Cada clase en Java debe pertenecer a un paquete, y se declara al principio de la clase:

```
package com.example.mi_paquete;

public class MiClase {
    // cuerpo de la clase
}
```

Para utilizar una clase de un paquete diferente, se debe importar:

```
import com.example.otro_paquete.OtraClase;

public class MiClase {
    OtraClase otraClase = new OtraClase();
}
```

En Kotlin, el sistema de paquetes es similar. La diferencia es que Kotlin no tiene un equivalente directo al `package-private` de Java; la visibilidad se controla de otra manera.

## 1.7 Orientación a objetos avanzada

### 1.7.1 Encapsulamiento

El encapsulamiento es uno de los pilares fundamentales de la programación orientada a objetos (OOP) y es una característica común tanto en Java como en Kotlin. El encapsulamiento se refiere a la habilidad de ocultar detalles de implementación de un objeto y exponer solo una interfaz pública. Esto permite proteger el estado interno del objeto de accesos no autorizados.

En Java, el encapsulamiento se logra mediante los modificadores de acceso: `public`, `private` y `protected`. Una variable declarada como `private` solo puede ser accedida dentro de la misma clase:

```
public class MiClase {
    private int variableDeInstancia;

    public int getVariableDeInstancia() {
        return variableDeInstancia;
    }

    public void setVariableDeInstancia(int variableDeInstancia) {
        this.variableDeInstancia = variableDeInstancia;
    }
}
```

En este ejemplo, la variable `variableDeInstancia` está declarada como privada y solo puede ser accedida a través de los métodos `get` y `set` que están declarados como públicos.

En Kotlin, el encapsulamiento se logra de manera similar:

```
class MiClase {
    private var variableDeInstancia: Int = 0
    get() = field
    set(value) {
        field = value
    }
}
```

```
}
```

En este ejemplo, la variable `variableDeInstancia` está declarada como privada y solo puede ser accedida a través del getter y setter generados automáticamente.

### 1.7.2 Getters y Setters

Los getters y setters son métodos utilizados para acceder y modificar variables de instancia de una clase, respectivamente. Estos métodos son comunes en ambos Java y Kotlin, y se utilizan para proporcionar una interfaz pública para acceder y modificar las variables mientras se mantiene el encapsulamiento.

En Java, los getters y setters son métodos explícitos. Los métodos get suelen tener el formato `getNombreDeVariable()` y los métodos set `setNombreDeVariable(valor)`:

```
public class MiClase {
    private int edad;

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        this.edad = edad;
    }
}
```

En Kotlin, al declarar una variable con `var`, los getters y setters se generan automáticamente:

```
class MiClase {
    var edad: Int = 0
}
```

También puedes crear getters y setters personalizados con lógica adicional:

```
class MiClase {
    private var _edad: Int = 0
    var edad: Int
        get() = _edad
        set(value) {
            // lógica adicional para modificar el valor
            _edad = value
        }
}
```

En Kotlin es posible definir un getter y un setter con una sola expresión, usando la variable temporal `field`:

```
class MiClase {
    var edad: Int = 0
    get() = field
    set(value) { field = value }
}
```

### 1.7.3 Herencia

En ambos lenguajes, las clases pueden heredar atributos y métodos de otras clases. En Java, se usa la palabra clave `extends`:

```
public class Student extends Person {
    private String major;
    private double GPA;

    public Student(String name, int age, String major, double GPA) {
        super(name, age); // Llamada al constructor de la clase padre
        this.major = major;
        this.GPA = GPA;
    }

    public String getMajor() {
        return major;
    }

    public void setMajor(String major) {
        this.major = major;
    }
}
```

En Kotlin, se utiliza la palabra clave `open` para permitir la herencia y los dos puntos para indicar la clase padre:

```
open class Person(var name: String, var age: Int)

class Student(name: String, age: Int, var major: String, var GPA: Double) : Person(name, age)
```

En Kotlin, las clases son `final` por defecto; hay que marcarlas como `open` si quieres que otras puedan heredar de ellas. En Java, las clases pueden heredar salvo que se declaren `final`.

### 1.7.4 Polimorfismo

El polimorfismo es un concepto de la POO que se refiere a la capacidad de utilizar objetos de distintas clases de forma siempre que compartan una interfaz común. Permite que una función o un método pueda ser utilizado con diferentes tipos de objetos sin conocer su clase específica.

Aquí hay un ejemplo de polimorfismo en Java:

```
class Figura {
    double area() {
        return 0;
    }
}

class Cuadrado extends Figura {
    double lado;
    Cuadrado(double lado) { this.lado = lado; }
```

```

    @Override
    double area() { return lado * lado; }
}

class Circulo extends Figura {
    double radio;
    Circulo(double radio) { this.radio = radio; }

    @Override
    double area() { return Math.PI * radio * radio; }
}

// Uso del polimorfismo
Figura cuadrado = new Cuadrado(10);
Figura circulo = new Circulo(5);
System.out.println(cuadrado.area()); // Imprime 100.0
System.out.println(circulo.area()); // Imprime 78.539...

```

En este caso, se han declarado tres clases: **Figura**, que es la clase padre, y **Cuadrado** y **Circulo**, que son las clases hijas. Todas las clases tienen un método **area()**, pero cada una lo implementa de forma diferente. Cuando se utiliza el polimorfismo, se pueden crear objetos de cualquiera de las clases hijas y utilizarlos como si fueran de la clase padre. Al llamar al método **area()** de cada objeto, se ejecuta la implementación específica de cada clase.

En Kotlin:

```

open class Figura {
    open fun area(): Double = 0.0
}

class Cuadrado(private val lado: Double) : Figura() {
    override fun area(): Double = lado * lado
}

class Circulo(private val radio: Double) : Figura() {
    override fun area(): Double = Math.PI * radio * radio
}

val cuadrado = Cuadrado(10.0)
val circulo = Circulo(5.0)
println(cuadrado.area()) // Imprime 100.0
println(circulo.area()) // Imprime 78.539...

```

En Kotlin, la herencia se declara con **open** y se usa **override** para sobrescribir métodos. El funcionamiento es el mismo que en Java.

### 1.7.5 Interfaces

En ambos lenguajes, las interfaces definen contratos que las clases deben implementar.

En Java:

```
public interface Comparable {  
    int compareTo(Object o);  
}
```

En Kotlin:

```
interface Comparable {  
    fun compareTo(o: Any): Int  
}
```

Una diferencia importante: en Java 8+, las interfaces pueden tener métodos con implementación por defecto (default methods). En Kotlin, esto es nativo con métodos de interfaz con cuerpo.

### 1.7.6 Clases abstractas

Las clases abstractas son clases que no se pueden instanciar directamente y deben ser extendidas por otras clases. Pueden tener métodos abstractos, que son métodos sin cuerpo que deben ser implementados por las clases que extienden la clase abstracta.

En Java:

```
abstract class Animal {  
    protected String name;  
  
    public Animal(String name) {  
        this.name = name;  
    }  
  
    public abstract void makeNoise();  
}  
  
class Dog extends Animal {  
    public Dog(String name) {  
        super(name);  
    }  
  
    @Override  
    public void makeNoise() {  
        System.out.println("Woof!");  
    }  
}
```

En Kotlin:

```
abstract class Animal(protected val name: String) {  
    abstract fun makeNoise()  
}  
  
class Dog(name: String) : Animal(name) {  
    override fun makeNoise() {  
        println("Woof!")  
    }  
}
```

```
}
```

La diferencia principal es la sintaxis: Kotlin usa `:` en lugar de `extends` y no necesita la palabra clave `abstract` en los métodos abstractos (aunque se puede usar).

### 1.7.7 Sobrecarga de métodos

En ambos lenguajes, es posible tener varias versiones de un mismo método con diferentes argumentos. Esto se conoce como sobrecarga de métodos.

En Java:

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public double add(double a, double b) {  
        return a + b;  
    }  
}
```

En Kotlin:

```
class Calculator {  
    fun add(a: Int, b: Int): Int {  
        return a + b  
    }  
  
    fun add(a: Double, b: Double): Double {  
        return a + b  
    }  
}
```

Kotlin también ofrece valores por defecto y argumentos nombrados, que pueden reducir la necesidad de sobrecarga:

```
fun add(a: Int, b: Int = 0): Int = a + b  
add(5)    // usa b = 0 por defecto  
add(5, 3) // b = 3
```

### 1.7.8 Sobrescritura de métodos

En ambos lenguajes, es posible modificar el comportamiento de un método heredado en una clase hija. En Java:

```
public class Shape {  
    public double area() {  
        return 0;  
    }  
}  
  
public class Circle extends Shape {
```

```

    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}

```

En Kotlin:

```

open class Shape {
    open fun area(): Double = 0.0
}

class Circle(private val radius: Double) : Shape() {
    override fun area(): Double = Math.PI * radius * radius
}

```

En Java, el modificador `@Override` es opcional pero recomendado; en Kotlin, `override` es obligatorio.

## 2 Java SE (Standard Edition)

### 2.1 Entrada y salida de datos

#### 2.1.1 Entrada

En Java, puedes leer datos de entrada del usuario a través del teclado utilizando el objeto `Scanner`. El objeto `Scanner` se encuentra en el paquete `java.util`, por lo que necesitarás importarlo:

```

import java.util.Scanner;

public class MiClase {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Ingresa un número entero: ");
        int numero = sc.nextInt();
        System.out.println("Has ingresado el número " + numero);
    }
}

```

En Kotlin puedes usar también `Scanner`:

```

import java.util.Scanner

fun main() {
    val sc = Scanner(System.`in`)
    print("Ingresa un número entero: ")
    val numero = sc.nextInt()
}

```

```
println("Has ingresado el número $numero")
}
```

Kotlin también ofrece funciones propias para lectura de datos. La más simple es `readln()`, que lee una línea completa:

```
fun main() {
    print("Ingresa un número entero: ")
    val entrada = readln()
    val numero = entrada.toIntOrNull()
    println("Has ingresado el número $numero")
}
```

La función `readlnOrNull()` devuelve `null` si no hay datos disponibles. `toIntOrNull()` convierte la cadena a entero o devuelve `null` si no es un número válido, lo cual es más seguro que `toInt()` que lanza una excepción si la cadena no es numérica.

Hay otras funciones como `readln()` (lanza excepción si no hay entrada) y en versiones anteriores existía `readLine()` y `readInt()`, que ya no son la forma recomendada.

### 2.1.2 Salida

En Java, puedes escribir datos de salida a la consola utilizando la clase `System.out`. Esta clase tiene varios métodos que te permiten imprimir diferentes tipos de datos:

```
public class MiClase {
    public static void main(String[] args) {
        int miEntero = 5;
        double miFlotante = 3.14;
        char miCaracter = 'a';
        boolean miBoolean = true;
        String miCadena = "Hola mundo!";

        System.out.println(miEntero);
        System.out.println(miFlotante);
        System.out.println(miCaracter);
        System.out.println(miBoolean);
        System.out.println(miCadena);
    }
}
```

En Kotlin:

```
fun main() {
    val miEntero = 5
    val miFlotante = 3.14
    val miCaracter = 'a'
    val miBoolean = true
    val miCadena = "Hola mundo!"

    println(miEntero)
    println(miFlotante)
}
```

```
println(miCaracter)
println(miBoolean)
println(miCadena)
}
```

En ambos casos, se utiliza `println()` para imprimir con salto de línea. También puedes usar `print()` para imprimir sin salto. La diferencia principal es que Java requiere que el método esté dentro de una clase, mientras que Kotlin permite funciones de nivel superior.

## 2.2 Excepciones

Las excepciones son eventos que ocurren durante la ejecución de un programa y que interrumpen el flujo normal. En Java y Kotlin, las excepciones se pueden utilizar para manejar errores de forma más controlada.

Aquí hay un ejemplo en Java:

```
public class Main {
    public static void main(String[] args) {
        try {
            int resultado = dividir(10, 0);
            System.out.println(resultado);
        } catch (ArithmeticException e) {
            System.out.println("Error: no se puede dividir por cero");
        } finally {
            System.out.println("Fin del programa");
        }
    }

    public static int dividir(int a, int b) {
        return a / b;
    }
}
```

Se ha declarado una función `dividir()` que divide dos números enteros. Si el segundo parámetro es cero, se lanzará una excepción de tipo `ArithmeticException`. El bloque `try-catch` permite capturar la excepción y ejecutar código específico si ocurre. El bloque `finally` se ejecuta siempre, independientemente de si ocurre una excepción o no.

En Kotlin, el uso de excepciones es similar, pero con una diferencia importante: las excepciones no son obligatorias. En Java, ciertas excepciones (las **checked exceptions**) deben ser declaradas en la firma del método o capturadas. En Kotlin, no existe este requisito:

```
fun main() {
    try {
        val resultado = dividir(10, 0)
        println(resultado)
    } catch (e: ArithmeticException) {
        println("Error: no se puede dividir por cero")
    } finally {
        println("Fin del programa")
    }
}
```

```

    }
}

fun dividir(a: Int, b: Int): Int = a / b

```

### 2.2.1 Excepciones a medida

En Java y Kotlin es posible crear excepciones a medida, definiendo nuevas clases de excepción ajustadas a las necesidades del programa.

En Java:

```

public class MiExcepcion extends Exception {
    public MiExcepcion(String mensaje) {
        super(mensaje);
    }
}

```

Se ha declarado una clase `MiExcepcion` que hereda de `Exception` y tiene un constructor que recibe un mensaje de error. Esta excepción se puede lanzar y capturar de la misma forma que cualquier otra excepción.

En Kotlin:

```

class MiExcepcion(mensaje: String) : Exception(mensaje)

```

La sintaxis es más concisa pero el resultado es equivalente.

## 2.3 Archivos y directorios

En ambos Java y Kotlin se puede trabajar con archivos utilizando las clases y métodos de las bibliotecas estándar.

Java proporciona varias clases en el paquete `java.io` y `java.nio.file` para trabajar con archivos. La clase `File` permite trabajar con información sobre un archivo o directorio, mientras que las clases `FileReader` y `FileWriter` permiten leer y escribir.

Ejemplo en Java que lee un archivo:

```

import java.io.*;

public class LeerArchivo {
    public static void main(String[] args) {
        try {
            File archivo = new File("archivo.txt");
            FileReader lector = new FileReader(archivo);
            BufferedReader buffer = new BufferedReader(lector);

            String linea;
            while ((linea = buffer.readLine()) != null) {
                System.out.println(linea);
            }
            buffer.close();
        }
    }
}

```

```

        lector.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

En Kotlin, la forma más cómoda es usando las extensiones de la clase `File`:

```

import java.io.*

fun main() {
    val archivo = File("archivo.txt")
    archivo.useLines { lines -> lines.forEach { println(it) } }
}

```

O también se puede usar `readText` y `writeText` para leer y escribir archivos completos:

```

val texto = File("archivo.txt").readText()
File("archivo.txt").writeText("Nuevo texto")

```

Para código Java moderno, `java.nio.file.Path` y `Files` son preferibles a construir toda la API con `java.io.File`:

```

import java.nio.file.*;
import java.io.*;

Path path = Path.of("students.txt");
try (BufferedReader reader = Files.newBufferedReader(path)) {
    for (String line; (line = reader.readLine()) != null;) {
        System.out.println(line);
    }
} catch (IOException exception) {
    System.out.println("Error: " + exception.getMessage());
}

```

El `try-with-resources` cierra automáticamente los recursos. El equivalente habitual en Kotlin es `use`:

```

Path.of("students.txt").bufferedReader().useLines { lines ->
    lines.forEach(::println)
}

```

## 2.4 Streams

Java Streams no son streams de bytes: son una abstracción para procesar secuencias de datos de forma declarativa. Java proporciona una serie de clases para trabajar con streams de bytes en el paquete `java.io`.

Por ejemplo, para leer bytes de un archivo:

```

import java.io.*;

try (FileInputStream fis = new FileInputStream("archivo.txt")) {

```

```

    int contenido;
    while ((contenido = fis.read()) != -1) {
        System.out.print((char) contenido);
    }
} catch (IOException e) {
    e.printStackTrace();
}

```

En Kotlin, se pueden usar las clases de streams de Java:

```

val inputStream = File("archivo.txt").inputStream()
inputStream.use {
    it.bufferedReader().forEachLine { println(it) }
}

```

Para procesamiento de datos, Java Streams (la API de `java.util.stream`) permite operaciones como filtrar, transformar y recopilar colecciones de datos:

```

import java.util.stream.*;

List<String> names = List.of("Ana", "Pedro", "María", "Juan");
List<String> resultado = names.stream()
    .filter(name -> name.length() > 4)
    .map(String::toUpperCase)
    .sorted()
    .toList();
// ["MARÍA", "PEDRO"]

```

En Kotlin, las colecciones tienen métodos similares integrados:

```

val names = listOf("Ana", "Pedro", "María", "Juan")
val resultado = names
    .filter { it.length > 4 }
    .map { it.uppercase() }
    .sorted()
// ["MARÍA", "PEDRO"]

```

## 2.5 Colecciones

Java y Kotlin ofrecen colecciones para almacenar y manipular grupos de objetos. Ambos lenguajes comparten la misma base en la JVM, pero la API que cada uno expone es diferente.

En Java, las colecciones principales son:

- **ArrayList**: lista dinámica, acceso rápido por índice.
- **LinkedList**: lista enlazada, inserción/eliminación rápida en extremos.
- **HashMap**: mapa clave-valor, acceso rápido por clave.
- **HashSet**: conjunto sin duplicados.

```

import java.util.*;

```

```

List<String> nombres = new ArrayList<>(List.of("Ana", "Pedro", "Juan"));
nombres.add("María");
nombres.remove("Pedro");

Map<String, Integer> edades = new HashMap<>();
edades.put("Ana", 25);
edades.put("Pedro", 30);

Set<String> colores = new HashSet<>(List.of("Rojo", "Verde", "Azul"));
colores.add("Rojo"); // no se añade: ya existe

```

En Kotlin, las colecciones se dividen en **inmutables** (`listOf`, `mapOf`, `setOf`) y **mutables** (`mutableListOf`, `mutableMapOf`, `mutableSetOf`):

```

val nombres = mutableListOf("Ana", "Pedro", "Juan")
nombres.add("María")
nombres.remove("Pedro")

val edades = mutableMapOf("Ana" to 25, "Pedro" to 30)

val colores = mutableSetOf("Rojo", "Verde", "Azul")
colores.add("Rojo") // no se añade: ya existe

```

Una diferencia importante: en Kotlin, `List` es una interfaz de solo lectura y `MutableList` permite modificaciones. En Java, `List` no garantiza que una implementación sea inmutable: `List.of` sí crea una lista no modificable, mientras que `new ArrayList` crea una mutable.

Para la mayoría de casos, `ArrayList` es la elección habitual en Java. `LinkedList` no es una mejora general ni ofrece acceso aleatorio eficiente; solo tiene sentido en casos concretos de inserciones y extracciones en extremos. Para claves, `HashMap` y `HashSet` dependen de que `equals` y `hashCode` estén bien implementados.

### 2.5.1 Operaciones de colección

Java Streams y Kotlin collections ofrecen operaciones funcionales similares: filtrar, transformar, ordenar, etc.

En Kotlin:

```

val resultado = nombres
    .filter { it.length > 3 }
    .map { it.uppercase() }
    .sorted()

```

En Java:

```

List<String> resultado = nombres.stream()
    .filter(nombre -> nombre.length() > 3)
    .map(String::toUpperCase)
    .sorted()
    .toList();

```

Un Java Stream no es un stream de bytes: es una secuencia perezosa de operaciones sobre datos.

Para una fuente que solo se recorre una vez puede ser más claro usar un `for`. No encadenes Streams solo para imitar Kotlin: usa la herramienta que mejor se adapte a cada caso.

## 2.6 Multithreading

Java y Kotlin soportan la ejecución concurrente de código, pero con modelos diferentes.

En Java, la concurrencia se basa en hilos (`Thread`) y en la API `java.util.concurrent`:

```
// Crear un hilo con una tarea
Thread hilo = new Thread(() -> {
    System.out.println("Ejecutando en hilo separado");
});
hilo.start();

// Usar ExecutorService para gestionar múltiples hilos
ExecutorService executor = Executors.newFixedThreadPool(2);
executor.submit(() -> System.out.println("Tarea 1"));
executor.submit(() -> System.out.println("Tarea 2"));
executor.shutdown();
```

Java 21 introdujo los **virtual threads** (hilos virtuales), que hacen económico tener muchas tareas bloqueantes:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> result = executor.submit(() -> {
        Thread.sleep(1000);
        return "Resultado";
    });
    System.out.println(result.get());
}
```

Los virtual threads son threads gestionados por la JVM que mantienen el modelo de `Thread`, `ExecutorService` y `Future`. Son una característica permanente desde Java 21.

En Kotlin, la concurrencia se basa en **coroutines**: cálculos suspendibles que pueden pausar y reanudar su ejecución. La mayoría de las APIs de coroutines pertenecen a `kotlinx.coroutines`, no al núcleo del lenguaje:

```
import kotlinx.coroutines.*

suspend fun cargarEstudiante(): Student = repository.load()

suspend fun cargarAmbos(): Pair<Student, Student> = coroutineScope {
    val first = async { cargarEstudiante() }
    val second = async { cargarEstudiante() }
    first.await() to second.await()
}
```

Una coroutine es un cálculo suspendible. Puede suspender sin bloquear el hilo y reanudar en otro, según su dispatcher. `launch` y `async` necesitan un `CoroutineScope`; no son constructores de hilos.

En ambos modelos hacen falta cancelación, límites de concurrencia, manejo de errores y una política

clara de acceso a estado compartido. La diferencia no es "Java lento y Kotlin rápido": es un modelo de ejecución y una API distinta.

## 2.7 Genéricos

Los genéricos permiten definir clases, interfaces y métodos que pueden trabajar con diferentes tipos de datos, proporcionando type safety en tiempo de compilación.

En Java:

```
public class Caja<T> {
    private T contenido;

    public Caja(T contenido) {
        this.contenido = contenido;
    }

    public T getContenido() {
        return contenido;
    }
}
```

En Kotlin:

```
class Caja<T>(private val contenido: T) {
    fun getContenido(): T = contenido
}
```

Una diferencia importante es la **variancia**. En Java se usan wildcards:

```
static void printAll(List<? extends Animal> source) {
    source.forEach(animal -> animal.makeNoise());
}

static void addAnimal(List<? super Animal> target) {
    target.add(new Dog("Rex"));
}
```

La regla práctica PECS es **Producer Extends, Consumer Super**: si solo lees de una colección usa ? **extends**, si solo escribes usa ? **super**.

En Kotlin se usan las palabras clave **out** (**extends**) e **in** (**super**):

```
fun printAll(source: List<out Animal>) {
    source.forEach { it.makeNoise() }
}

fun addAnimal(target: MutableList<in Animal>) {
    target.add(Dog("Rex"))
}
```

Kotlin también permite proyecciones en el uso. La idea fundamental es la misma: no existe en Java una `List<String>` que sea subtipo de `List<Object>` porque permitiría insertar un objeto no `String`.

## 3 Java 25: qué es nuevo y qué enseñar

### 3.1 Características permanentes de Java 25

Java 25 incluye varias características que ya son permanentes (no preview):

- **Module import declarations (JEP 511):** permite importar todos los tipos exportados por un módulo con una sola declaración. Es útil en ejemplos docentes, pero no sustituye a entender paquetes e imports normales.
- **Compact source files and instance main methods (JEP 512):** simplifica pequeños programas docentes. Java 25 proporciona la clase `I0` en `java.lang` para entrada/salida textual de estos fuentes compactos.
- **Flexible constructor bodies (JEP 513):** permite determinadas sentencias antes de la invocación explícita a `super` en constructores. Es una mejora avanzada y no una razón para reescribir constructores sencillos.

Estas características son de Java 25, pero no todas deben entrar en la primera práctica. La sintaxis tradicional sigue siendo válida y suele ser más transferible entre versiones y herramientas.

### 3.2 Características modernas anteriores que sí conviene conocer

Además de las novedades de Java 25, conviene conocer estas características que ya forman parte del Java moderno:

- `var` local, desde Java 10.
- expresiones `switch` con flechas, desde Java 14.
- text blocks (`"""`), desde Java 15.
- pattern matching con `instanceof`, records y sealed classes, desde Java 16/17.
- record patterns y pattern matching con `switch`, permanentes desde Java 21.
- virtual threads, permanentes desde Java 21.
- colecciones secuenciadas (`getFirst`, `getLast`, `reversed`), desde Java 21.

### 3.3 Preview de Java 25

JEP 507, **Primitive Types in Patterns, instanceof, and switch**, es preview en Java 25. Para usarlo hay que compilar y ejecutar con `--enable-preview` y la versión correspondiente del JDK. Los ejemplos de esta guía no dependen de él.

### 3.4 Lo que no se debe presentar como Java 25

No son novedades de Java 25: records, sealed classes, pattern matching básico, virtual threads, text blocks ni expresiones `switch`. Son características de versiones anteriores que siguen siendo importantes en Java 25.

## 4 Java web: frontera de esta guía

Esta comparación termina en el lenguaje y las APIs esenciales de la JVM. Servlets, JSP, Jakarta Persistence, JTA, JMS, Spring MVC y Spring Boot deben explicarse en documentos de desarrollo web separados, donde se puedan fijar versiones, dependencias y ejemplos ejecutables.

En particular:

- Jakarta EE no debe describirse como "Java EE" actual.
- `jakarta.persistence.*` no es el mismo paquete que el antiguo `javax.persistence.*`.
- Exposed es una biblioteca Kotlin de SQL/DAO, no una implementación de JPA.
- Una sección con headings vacíos o código incompleto no aporta una ruta de aprendizaje; es preferible enlazar al material de la unidad web.

## 5 Actividades propuestas

### 5.1 Actividad 1: traducir el modelo

Implementa `Student` primero como clase Java mutable con getters y setters, después como `record`. Compara construcción, acceso, igualdad y validación.

### 5.2 Actividad 2: resultado sellado

Modela `Result` con `Success` y `Failure`. Escribe una función Kotlin con `when` y otra Java con `switch` sobre patrones. Añade un caso y observa qué debe cambiar.

### 5.3 Actividad 3: colección idiomática

Parte de una `List<Student>`, filtra por curso y obtiene nombres ordenados. Escribe una versión con un bucle y otra con `Stream`. Justifica cuál es más legible, no solo cuál tiene menos líneas.

### 5.4 Actividad 4: frontera null

Expón desde Java un método que pueda recibir `null` y úsalo desde Kotlin. Documenta la API con anotaciones de nulabilidad y valida el argumento en el punto de entrada.

### 5.5 Actividad 5: concurrencia con criterio

Ejecuta dos cargas independientes con `ExecutorService` y `virtual threads`. Compara el diseño con una versión Kotlin basada en `coroutineScope` y `async`. Mide tiempos solo después de comprobar que ambas soluciones hacen el mismo trabajo.

## 6 Referencias oficiales

- Java SE 25, cambios del lenguaje por versión: <https://docs.oracle.com/en/java/javase/25/language/java-language-changes-release.html>
- Java SE 25, características preview: <https://docs.oracle.com/en/java/javase/25/language/preview-language-vm-features.html>

- JEP 511, module import declarations: <https://openjdk.org/jeps/511>
- JEP 512, compact source files and instance main methods: <https://openjdk.org/jeps/512>
- JEP 513, flexible constructor bodies: <https://openjdk.org/jeps/513>
- JEP 507, primitive types in patterns (preview): <https://openjdk.org/jeps/507>
- Kotlin, comparación con Java: <https://kotlinlang.org/docs/comparison-to-java.html>
- Kotlin, null safety: <https://kotlinlang.org/docs/null-safety.html>
- Kotlin, colecciones: <https://kotlinlang.org/docs/collections-overview.html>
- Kotlin, coroutines básicas: <https://kotlinlang.org/docs/coroutines-basics.html>
- Kotlin, interoperabilidad con Java: <https://kotlinlang.org/docs/java-interop.html>